

Today's Agenda:-

1. Truth Table
2. Basic AND / OR / XOR properties
3. Left Shift & Right Shift Operators
4. Signle Number
5. Signle Number 2

Starting 7:05

Bit-wise Operators : $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$

a	b	$a\&b$	$a b$	$a\wedge b$	$\sim a$
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Basic Properties

1. Even / Odd Number →

$A \& 1$ → 0 (A is even)
→ 1 (A is odd)

a) $A = 10$

$$\begin{array}{r} 1010 \\ \& 0001 \\ \hline 0000 \end{array}$$

b) $A = 11$

$$\begin{array}{r} 1011 \\ \& 0001 \\ \hline 0001 \end{array}$$

2. $A \& 0 \rightarrow 0$

Let $A = 11$

$$\begin{array}{r} 1011 \\ \& 0000 \\ \hline 0000 \end{array}$$

$$3. \quad A \& A \rightarrow \underline{A}:$$

$$\text{Let } A = 11$$

$$\begin{array}{r} 1011 \\ \times 1011 \\ \hline 1011 \end{array}$$

$$4. \quad A|0 \rightarrow \underline{A}:$$

$$\text{Let } A = 11$$

$$\begin{array}{r} 1011 \\ | 0000 \\ \hline 1011 \end{array}$$

$$5. \quad A|A \rightarrow \underline{A}:$$

$$\text{Let } A = 11$$

$$\begin{array}{r} 1011 \\ | 1011 \\ \hline 1011 \end{array}$$

$$6. \quad A^0 \rightarrow \underline{A}.$$

$$\text{Let } A = 11$$

$$\begin{array}{r} 1011 \\ \wedge 0000 \\ \hline 1011 \\ \hline \end{array}$$

$$7. A \wedge A \rightarrow 0.$$

$$\text{Let } A = 11$$

$$\begin{array}{r} 1011 \\ \wedge 1011 \\ \hline 0000 \end{array}$$

Commutative Property $\rightarrow$ Order of the operands does not affect the result of the bitwise operator.
ex:- $+$, $\times$

$$(a \& b) = (b \& a)$$

$$(a | b) = (b | a)$$

$$(a \wedge b) = (b \wedge a)$$

$-$, $!$, $\sim$ are not.

Associative Property → Grouping of operands does not affect the result of a bitwise operator.

$$a \& (b \& c) = (a \& b) \& c$$

$$a | (b | c) = (a | b) | c$$

$$a \wedge (b \wedge c) = (a \wedge b) \wedge c$$

ex:- $3 \& (4 \& 5) = (3 \& 4) \& 5$

$$\begin{array}{r}
 1004 \\
 \& 1015 \\
 \hline
 100 \\
 \& 0113 \\
 \hline
 000
 \end{array}$$

$$\begin{array}{r}
 0113 \\
 \& 1004 \\
 \hline
 000 \\
 \& 1015 \\
 \hline
 000
 \end{array}$$

Q-1.

< Question- 1 > : Evaluate the expression: $a \wedge b \wedge a \wedge d \wedge b$

$$\begin{aligned} &= \underline{a \wedge a} \wedge \underline{b \wedge b} \wedge \underline{d} \\ &= \underline{0 \wedge 0} \wedge d \\ &= 0 \wedge d \\ &= d \wedge 0 = d \end{aligned}$$

< Question- 2 > : Evaluate the expression: $1 \wedge 3 \wedge 5 \wedge 3 \wedge 2 \wedge 1 \wedge 5$

$$\begin{aligned} &= \underline{1 \wedge 1} \wedge \underline{3 \wedge 3} \wedge \underline{5 \wedge 5} \wedge \underline{2} \\ &= 0 \wedge 0 \wedge 0 \wedge 2 \\ &= 0 \wedge 0 \wedge 2 \\ &= 0 \wedge 2 = 2 \end{aligned}$$

$$\begin{array}{r} 061 \\ \wedge 011 \\ \hline 010 \\ \wedge 101 \\ \hline 111 \\ \wedge 011 \\ \hline 100 \\ \wedge 010 \\ \hline 110 \\ \wedge 001 \\ \hline 111 \\ \wedge 101 \\ \hline 010 \end{array}$$

$$010 = 2$$

Q-3.

$$\begin{aligned} &120 \wedge 5 \wedge 6 \wedge 6 \wedge 120 \wedge 5 \\ &= \underline{120 \wedge 120} \wedge \underline{5 \wedge 5} \wedge \underline{6 \wedge 6} \\ &= 0 \wedge 0 \wedge 0 = 0 \end{aligned}$$



Left Shift Operator ($\ll$)

It shifts the bits of a number to the left by specified number of positions.

a =	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>1</u>	<u>0</u>	<u>1</u>	<u>0</u>	10
a << 1	<u>0</u>	<u>0</u>	<u>0</u>	<u>1</u>	<u>0</u>	<u>1</u>	<u>0</u>	<u>0</u>	$20 = 10 \times 2^1$
a << 2	<u>0</u>	<u>0</u>	<u>1</u>	<u>0</u>	<u>1</u>	<u>0</u>	<u>0</u>	<u>0</u>	$40 = 10 \times 2^2$
a << 3	<u>0</u>	<u>1</u>	<u>0</u>	<u>1</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	$80 = 10 \times 2^3$
a << 4	<u>1</u>	<u>0</u>	<u>1</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	$= 160$
a << 5	<u>—</u>	<u>—</u>	<u>—</u>	<u>—</u>	<u>—</u>	<u>—</u>	<u>—</u>	<u>—</u>	

↓
overflow
condition

$$a \ll N = a \times 2^N$$

Replacing by 1.

$$1 \ll N = 2^N$$

-128 to 127.

Right Shift Operator ($\gg$) Shifts the bit of a number to its right by specified positions

a =	0	0	0	0	1	0	1	0	= 10
a >> 1	0	0	0	0	0	1	0	1	= 5 = $10/2^1$
a >> 2	0	0	0	0	0	0	1	0	= 2 = $10/2^2$
a >> 3	0	0	0	0	0	0	0	1	= 1 = $10/2^3$
a >> 4	0	0	0	0	0	0	0	0	= 0 = $10/2^4$

$$a \gg N = a / 2^N$$

Q:- 4

$$1 \ll 3 = 2^3 = 8$$

0 0 0 1

0 0 1 $\swarrow$ 0 = 1 << 1 = 2

0 1 $\swarrow$ 0 $\swarrow$ 0 = 1 << 2 = 4

$\swarrow$ $\swarrow$ $\swarrow$ = 1 << 3 = 8

1 0 0 0

Power of Left Shift Operator

1. **OR Operator** $\rightarrow$ **SETS** the i^{th} bit of a number.

ex:

$$45 \mid (1 \ll 4)$$

	5	4	3	2	1	0	
	1	0	1	1	0	1	= 45
		1	0	0	0	0	= $1 \ll 4$
							which is 16
	1	1	1	1	0	1	
---	1	1	1	1	0	1	

$$a \mid (1 \ll i)$$

$$45 \mid (1 \ll 3)$$

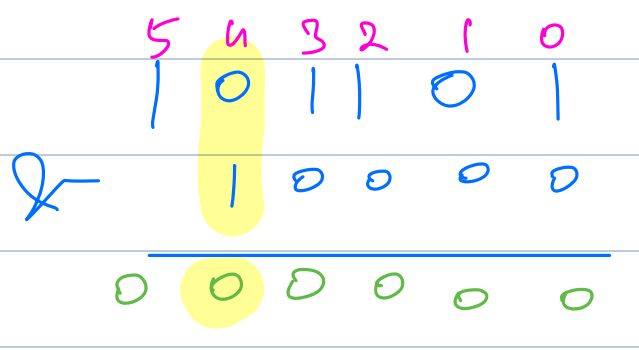
Set $\rightarrow 1$
Unset $\rightarrow 0$

	1	0	1	1	0	1
			1	0	0	0
	1	0	1	1	0	1
---	1	0	1	1	0	1

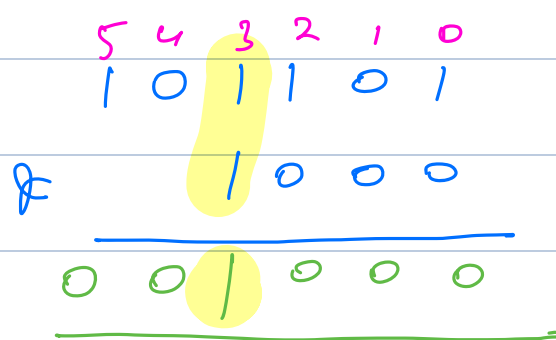
Sets the i^{th} bit of a no.
if unset, keep it same if
already set.

2. **AND Operator** $\rightarrow$ Checks whether the i^{th} bit of a no. is set or not set.

$$45 \& (1 << 4)$$



$$45 \& (1 << 3)$$



$$A \& (1 << i)$$

0 (i^{th} bit is Unset)

2^i (i^{th} bit is set).

Flip

3. XOR Operator $\rightarrow$ Toggles the bit at the j^{th} index.

$$45 \wedge (1 \ll 4)$$

$$\begin{array}{r} 101101 \\ \wedge 100000 \\ \hline 111101 \end{array}$$

$$45 \wedge (1 \ll 3)$$

$$\begin{array}{r} 101101 \\ \wedge 010000 \\ \hline 100101 \end{array}$$



< Question > : Check whether ith bit is set or not.

① $N = 35$; $i = 5$

5	4	3	2	1	0
1	0	0	0	1	1

true.

② $N = 55$; $i = 3$

5	4	3	2	1	0
1	1	0	1	1	1

false.

boolean checkBit (int N, int i) {

if ($N \& (1 << i) == 0$) {

return false

} else {

return true;

}

}

T.C. $\rightarrow O(1)$

Q:- UNSET the i^{th} bit of a number if it is set.

ex:- $N = 6$; $i = 2$

2	1	0
1	1	0

so there it is already set
so we need to unset it



0 1 0 $\rightarrow$ 2

ex:- $N = 10$; $i = 2$;

3	2	1	0
1	0	1	0

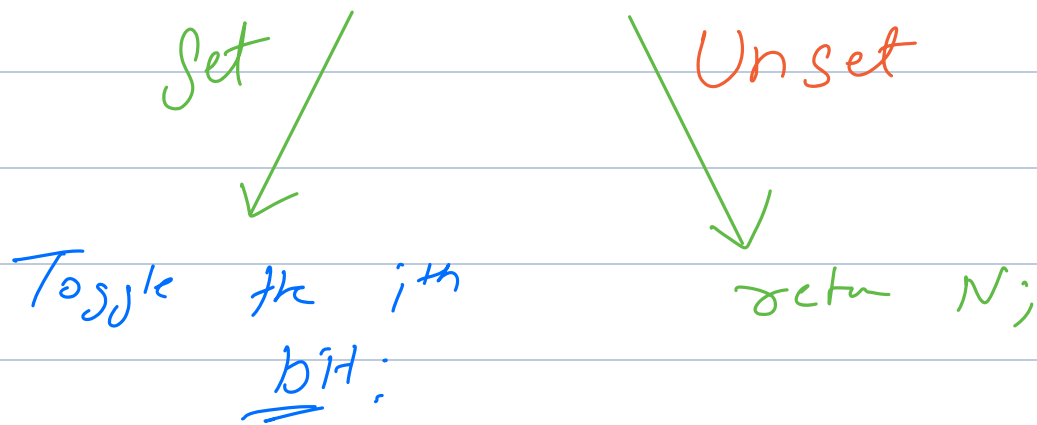
here it is already unset(0)
so answer is as number



1010 = 10

Approach:-

1. Check whether the i^{th} bit is set or not set.



Code:-

```
int unset (int N, int i) {  
    if (checkBit (N, i) == true) {  
        return N ^ (1 << i);  
    } else {  
        return N;  
    }  
}
```

T.C $\rightarrow$ $O(1)$

8:35

1. Given an array of size N . where every element occurs twice except one element. Identify that element.

ex:- {4, 5, 5, 4, 1, 6, 6}

ans = 1.

ex:- {7, 5, 5, 1, 7, 6, 1, 6, 4}

ans = 4.

Approach Apply the property of $A \oplus A = 0$ that cancels out every no that occurs in pair. The one isolated will be left. Do a XOR on all the nos.

Code

```
int findUnique (int arr[], int N) {  
    int ans = 0;  
    for (i = 0; i < N; i++) {  
        ans = ans ^ arr[i];  
    }  
    return ans;  
}
```

T.C $\rightarrow O(N)$

Approach 2:-

Ex:- { 2, 3, 5, 6, 3, 6, 2 }

i

	2	1	0	ans = 0 1
2 →	0	1	0	ans / (1 < 0)
3 →	0	1	1	
5 →	1	0	1	ans =
6 →	1	1	0	ans / (1 < 2)
3 →	0	1	1	1 / (1 < 2)
6 →	1	1	0	001
2 →	0	1	0	100
	2	6	2	101
	+1	+0	+1	

for the time wine thing there is not 5 and we take summation of all the set bits so ans will be = 262

5 =

0000 0000 0000 0000

Idea:- For every position, count the no.s which have the bit on that position as set.

if (count is even)

i^{th} bit is unset in the no. I am looking for.

else

i^{th} bit is set in the no. I am looking for. SET the i^{th} bit.

$$\text{ans} = \text{ans} | (1 \ll i)$$

Code

```
int findUnique (int arr[], int N) {  
    int ans = 0;  
    for (i = 0; i < 32; i++) {  
        int count = 0;  
        for (j = 0; j < N; j++) {  
            if (CheckBit(arr[j], i)  
                == true) {  
                count++;  
            }  
        }  
        if (count % 2 == 1) {  
            ans = ans | (1 << i);  
        }  
    }  
    return ans;  
}
```

T.C $\rightarrow O(N)$

3. Every element occurs twice except two elements. Identify the 2 elements.

ex:- {4, 5, 4, 5, 2, 1}

2, 1.

ex:- {4, 5, 4, 1, 1, 6, 5, 2}

6, 2.

Approach:-

ex:- {4, 5, 4, 1, 1, 6, 5, 2}

① Take XOR of everything

$4 \wedge 5 \wedge 4 \wedge 1 \wedge 1 \wedge 6 \wedge 5 \wedge 2$

$$= 6 \wedge 2 = \begin{array}{r} 110 \\ 010 \\ \hline \end{array}$$

4.

100

2	1	0
1	0	0

G_{n-1}
2th bit
set

G_{n-2}
2th bit
unset

	2	1	0
4	1	0	0
5	1	0	1
4	1	0	0
1	0	0	1
1	0	0	1
6	1	1	0
5	1	0	1
2	0	1	0

first = 0
 ^ 4
 ^ 5
 ^ 4
 ^ 6
 ^ 5
 = 6.

second = 0
 ^ 1
 ^ 1
 ^ 2
 = 2.

ex:- { 4, 5, 4, 5, 2, 1 }

= 4 ^ 5 ^ 4 ^ 5 ^ 2 ^ 1 = $\begin{array}{r} 010 \\ 001 \\ \hline 011 = 3 \end{array}$

1th bit
Set

1th bit
Unset

first = 0
 ^ 2

second = 0
 ^ 4
 ^ 5

$$= \underline{\underline{2}}$$

$$\begin{array}{r} 14 \\ 15 \\ 16 \\ \hline = 1 \end{array}$$

① Take XOR of all the numbers

N { `int xorAll = 0;`
`for (i = 0; i < N; i++) {`
`xorAll = xorAll ^ arr[i];`
`}`

position of

② Identify the pos. of the 1st set bit in xorAll.

32 { `int pos = 0;`
`for (i = 0; i < 32; i++) {`
`if (checkBit (xorAll, i)`
`== true) {`
`pos = i;`
`break;`
`}`
`}`

③ Segregate entire array into 2 groups.

G₁ → pos bit is set.

G₂ → pos bit is unset.

```
int first = 0; second = 0;
for (i = 0; i < N; i++) {
    if (checkBit(arr[i], pos) == true) {
        first = first ^ arr[i];
    } else {
        second = second ^ arr[i];
    }
}
print(first);
print(second);
```

(Note: A green bracket labeled 'N' is drawn next to the for loop, and a blue bracket labeled '3' is drawn next to the if-else block.)

T.C → $O(N)$

S.C → $O(1)$

